

Desenvolvimento de conceitos comportamentais de funções de ordem superior

Shriram Krishnamurthi
Universidade Brown
Providence, RI, EUA

Kathi Fisler
Universidade Brown
Providence, RI, EUA

RESUMO

Motivação. Funções de ordem superior são um componente padrão e cada vez mais central em muitos tipos de programação moderna, incluindo ciência de dados e desenvolvimento web. No entanto, poucas pesquisas têm sido dedicadas ao aprendizado ou à compreensão desse tópico por parte dos alunos.

Objetivos. Realizamos uma pesquisa exploratória sobre a capacidade dos alunos de correlacionar funções de ordem superior com seu comportamento de entrada e saída. Também buscamos avaliar diversas técnicas para mensurar sua compreensão.

Método. Criamos uma série de instrumentos nos quais os alunos recebiam exemplos concretos de entrada/saída ou diagramas abstratos de transformações de listas. Os alunos foram solicitados a agrupar ou classificar esses exemplos de acordo com seu comportamento, às vezes em relação a uma lista concreta de funções de ordem superior e às vezes de forma livre. Administramos esses medicamentos ao longo de um mês e, em seguida, novamente três meses depois.

Resultados. Constatamos que os alunos inicialmente apresentam diversas dificuldades com o agrupamento de exemplos de funções de ordem superior. Com diferentes instrumentos, observamos que, posteriormente, os alunos conseguem ter um bom desempenho, evitando em grande parte erros de grande escala, mas cometendo vários erros de pequena escala. Também encontramos indícios de evolução no raciocínio deles em relação a essas operações. Além disso, identificamos fragilidades na natureza e na ordem das técnicas utilizadas.

Discussão. Funções de ordem superior merecem muito mais atenção do que têm recebido na literatura sobre ensino de programação. Seu uso crescente em diversos domínios importantes torna essa necessidade crucial. Os métodos que propomos para conduzir tal pesquisa são mais uma contribuição deste trabalho. Nossos resultados e métodos também devem ser relevantes para explorar como os alunos compreendem bibliotecas e APIs.

CONCEITOS CCS

• **Computação aplicada** y **Educação**.

PALAVRAS-CHAVE

funções de ordem superior, comportamento, agrupamento, classificação

Formato de referência da ACM:

Shriram Krishnamurthi e Kathi Fisler. 2021. Desenvolvimento de conceitos comportamentais de funções de ordem superior. In Anais da 14ª Conferência Internacional da ACM sobre Pesquisa em Educação Computacional (ICER 2021), 16 a 19 de agosto de 2021, Evento Virtual, EUA. ACM, Nova York, NY, EUA, 13 páginas. <https://doi.org/10.1145/3446871.3469739>

1 INTRODUÇÃO

Funções de ordem superior (hofs) estão por toda parte na programação.

O que começou como um recurso de nicho em linguagens funcionais, como Lisp, agora é onipresente. O Java, a partir do Java 8, adicionou-os e os tipos de dados correspondentes (ou seja, Streams), para que os desenvolvedores Java possam (e de fato o façam) escrever código como

```
listOfStrings.stream() .filter(s ->
    s.contains("CS")) .collect(Collectors.toList)
```

Desenvolvedores web costumam escrever código

```
como $
("ul").filter( function() { return $(this).length == 2; }) .css("color", "red")
```

em jQuery, uma biblioteca JavaScript popular. Programadores Python que usam Pandas, uma biblioteca de análise de dados extremamente popular, também escrevem rotineiramente.

```
df['withTax'] = df['price'].apply(lambda x:x*1.05)
```

Programadores R em análise de dados frequentemente escrevem

```
Reduzir(f = "+", x = 1:6, acumular = FALSO)
```

Todos esses processos utilizam os ciclos HAF (filtrar, coletar, aplicar, reduzir).

É necessário esclarecer alguns termos: existem dois conceitos relacionados aqui, funções anônimas e funções de ordem superior. As primeiras (geralmente chamadas de "lambdas" em muitas linguagens, inclusive no Excel [16]) criam valores de função sem nomes. As últimas são funções que aceitam valores de função. Naturalmente, elas possuem uma simbiose estreita: as primeiras criam valores que as últimas podem aceitar. No entanto, funções de ordem superior podem existir sem funções anônimas. De fato, algumas progressões de aprendizado passam funções nomeadas para funções de ordem superior antes de ensinar funções anônimas. Nosso foco neste artigo são as funções de ordem superior, independentemente de como as funções que lhes são passadas foram criadas.

Existem muitos propósitos para os quais se pode usar hofs, e sua pedagogia deve corresponder a esses propósitos. Nosso foco aqui é em hofs para processamento de dados. Esse foco é compartilhado por diversos setores. Os exemplos acima, de R e Python, são ambos retirados de código de processamento de dados. Hadley Wickham, Cientista-Chefe do RStudio e um influente educador em ciência de dados, dedica um capítulo de Advanced R [29] a "Operadores de Função" e começou a criar vídeos [30] para apresentar hofs a programadores de R. Outro exemplo famoso de hofs em processamento de dados em larga escala é o operador MapReduce [5], que foi diretamente inspirado pelos hofs padrão map e reduce, e consome "funções" que implementam as etapas de mapeamento e redução. A crescente importância da ciência de dados e seu uso em

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que as cópias contenham este aviso e a citação completa na primeira página. Os direitos autorais dos componentes desta obra que pertencem a terceiros que não o (s) autor(es) devem ser respeitados. É permitido fazer resumos com a devida atribuição. Para copiar, republicar, postar em servidores ou redistribuir para listas, é necessária permissão prévia específica e/ou o pagamento de uma taxa. Solicite permissões pelo e-mail permissions@acm.org. ICER 2021, 16 a 19 de agosto de 2021, Evento Virtual, EUA © 2021 Direitos autorais reservados ao(s) proprietário(s)/autor(es). Direitos de publicação licenciados à ACM. ACM ISBN 978-1-4503-8326-4/21/08...\$15,00 <https://doi.org/10.1145/3446871.3469739>

A aprendizagem de máquina, portanto, destaca a importância e a urgência de métodos pedagógicos de qualidade para os alunos. Essa necessidade generalizada não tem sido acompanhada pela pedagogia. Há pouca literatura sobre como ensinar efetivamente com recursos de apoio à aprendizagem. Mesmo livros introdutórios que abordam o tema de forma significativa (por exemplo, [1, 8, 26]) não foram avaliados quanto à sua eficácia nessa área.

Durante décadas, os autores deste artigo ensinaram funções de ordem superior (hofs) fazendo com que os alunos escrevessem percursos de processamento de dados como funções recursivas sobre listas (sem hofs), apontando então as semelhanças nas funções para tarefas semelhantes e abstraindo sobre os percursos semelhantes para definir hofs do zero. A partir disso, eles passam a mostrar a coleção de hofs embutida na linguagem de programação que está sendo ensinada e, em seguida, pedem aos alunos que definam e usem hofs durante o restante do curso. Essa abordagem é baseada na filosofia da seção Abstração de Como Projetar Programas [8]. Nessa abordagem, fundamentalmente, as hofs surgem como abstrações de código.

A seleção e aplicação de funções de busca de objetos (HFOs) na prática, no entanto, partem não do código de percurso existente, mas de uma descrição do comportamento desejado do programa. Essa descrição pode ser em prosa ou na forma de um exemplo de entrada/saída (ou caso de teste). Assim, para entender se nossa técnica pedagógica padrão está funcionando, precisamos estudar se o aprendizado sobre HFOs como abstrações de código ajuda os alunos a percebê-las como abstrações de comportamento definidas por características como tipos de entrada/saída e as relações entre as listas de entrada e as listas ou valores produzidos como saída.

Este artigo relata um estudo em várias etapas, no qual fornecemos aos alunos diversos exemplos concretos de entrada/saída de operações em listas e pedimos que os associassem a comportamentos suportados pelos hofs descritos na seção 4. Nossos dois primeiros estudos pediram aos alunos que agrupassem os exemplos por seus comportamentos e nomeassem os grupos. Nossos três últimos estudos pediram aos alunos que classificassem ou rotulassem os mesmos exemplos com hofs específicos que correspondessem a cada um. Os quatro primeiros estudos ocorreram logo após os alunos serem expostos aos hofs por meio da pedagogia padrão (abstração de código). O quinto ocorreu três meses depois, após os alunos terem usado os hofs em várias tarefas. Nossa análise se concentra em duas questões de pesquisa:

RQ 1. Quais características comportamentais dos HAFs (Helping Frameworks) os alunos percebem após aprenderem sobre eles como abstrações de código e escreverem pequenos programas utilizando-os?

RQ 2. Quais padrões comportamentais de transformações de listas os alunos associam corretamente aos nomes de hofs que exibem padrões correspondentes?

Analisamos esses dados pedindo aos alunos que agrupem e classifiquem exemplos de comportamento de funções, apresentados textualmente e diagramáticos. Em seguida, comparamos suas anotações com uma verdade fundamental definida por nós. Essa metodologia, que possui ampla aplicabilidade, é em si uma contribuição deste trabalho. Constatamos que eles conseguem distinguir algumas características melhor do que outras. A Seção 8 apresenta algumas reflexões sobre este trabalho.

2. FUNDAMENTAÇÃO TEÓRICA

Ao pedir aos alunos que rotulem exemplos, agrupem exemplos e nomeiem agrupamentos, estamos pedindo que eles formem abstrações. Os alunos já haviam trabalhado com estruturas de informação de alto nível (HFAs) como abstrações sobre programas semelhantes.

mas agora nos pediram para trabalhar com eles como abstrações sobre in-

Comportamento de entrada/saída.

As teorias de formação de abstrações baseiam-se em teorias da percepção [14, 15]: é preciso identificar características de uma coleção de amostras e discernir quais são importantes antes de abstrair sobre características comuns. No caso de hofs, numa perspectiva de entrada/saída, as características importantes incluem os tipos, comprimentos e ordem relativa entre entradas e saídas. A Figura 1 resume essas características em detalhe para os hofs utilizados neste estudo. O livro didático que os alunos leram inicialmente para aprender sobre hofs (antes do início do nosso estudo) faz referência a essas características, mas não as contrasta tão nitidamente quanto na tabela (que fornecemos aos alunos antes da terceira etapa do nosso estudo). Nosso estudo explora a identificação de características por meio dos rótulos que os alunos atribuem aos exemplos.

As principais características comportamentais que identificamos sobre os hofs são relacionais, pois capturam relações entre as entradas e saídas de uma computação. Gentner e Kurtz [13] propuseram categorias relacionais como uma forma fundamental de abstração, separada de outras formas baseadas na estrutura compartilhada de objetos (como trechos de código comuns). A hipótese do status da categoria propõe que as categorias relacionais auxiliam na recuperação cognitiva de conhecimento estruturalmente relevante, com base em tarefas que envolvem rotular, resumir e descrever materiais [20]. Nosso estudo emprega tarefas semelhantes como um primeiro passo para o desenvolvimento de categorias relacionais.

Embora as teorias de como os programadores desenvolvem esquemas [21, 24] também possam ser aplicadas ao nosso trabalho, as teorias de formação de abstração são mais relevantes. Os esquemas são padrões de código parcialmente instanciados. Elas seriam relevantes para este estudo se pedíssemos aos alunos que implementassem as travessias subjacentes várias vezes e (mentalmente) recuperassem os padrões dessas travessias (como acontece com a abstração de código).

Aqui, estamos mais preocupados com os alunos em recuperar o nome de um Hof apropriado com base no comportamento de entrada/saída. O objetivo principal de usar um Hof é que a estrutura de código comum não apareça explicitamente no programa resultante. Embora esquemas possam ser desenvolvidos ao longo do processo, nossa ênfase está na abstração comportamental, não na abstração do código.

3 TRABALHOS RELACIONADOS

Embora existam inúmeras fontes que ensinam HAFs (Habilidades de Ordenação de Habilidades), não temos conhecimento de literatura sobre a compreensão que os alunos têm delas. Da mesma forma, não encontramos trabalhos anteriores sobre como os alunos aprendem ou percebem as relações entre abstrações de código e abstrações comportamentais. Já foram realizados estudos que analisaram a seleção e o uso por parte dos alunos.

de hofs no contexto da composição de planos. O estudo interlinguístico de Fisler sobre o problema Rainfall [9] relatou o uso bem-sucedido de filter (e ocasionalmente fold) pelos alunos. O estudo relacionado de Fisler et al. [10] incluiu alunos que estavam aprendendo OCaml: eles usaram com sucesso tanto filter quanto take nos problemas do estudo.

Os padrões de projeto abstraem e nomeiam descritivamente tarefas comuns de programação [4, 12]. Em relação a este projeto, vemos os padrões de projeto como sendo semelhantes aos esquemas (discutidos na seção 2), no sentido de que o objetivo é aprender um padrão de código de várias linhas (ou expressão) em vez do nome de uma única operação integrada com um comportamento específico de entrada/saída.

A seleção de funções com base no comportamento desejado surge ao trabalhar com APIs. O contexto da API é um pouco diferente do contexto do HoF:

Funções\	Crítérios OT OET/I	A lista	SIM/SIM	WHE/I	Por tipo
filtro	OL/I pode ser diferente. Mesma		mesmo 1		A -> B
de mapa	lista, mesma, mesma. 1	<=			A -> Bool
levar um tempo	lista igual igual 1 até critério, depois 0	A -> Bool			
dobra	booleano -N/A-	const 1	-N/A- 1 até verdadeiro, depois 0		A -> Bool
ormap	qualquer pode diferir qualquer prefixo/sufixo				AB -> B
onde os cabeçalhos das colunas são indicados pela seguinte legenda:					
Abreviação	Expansão/Significado				
OT	Tipo de saída				
AINDA/EU	Para listas de saída, o tipo de elemento de saída é relativo à entrada.				
ELES SÃO	Comprimento da saída em relação à entrada				
SIM/SIM	Ordem de saída em relação à entrada				
WHE/I	Quais elementos de entrada, e quantos deles, determinam um elemento de saída?				
Por tipo	Tipo de operação consumida pelo hof				

Figura 1: Resumo das características do hof

As APIs geralmente mantêm estado (portanto, os tipos são resumos insuficientes do comportamento) e suportam conceitos de domínio além daqueles com os quais o programador já trabalha (por exemplo, acesso a um...). serviço em nuvem versus extrair itens de uma lista). Os programadores frequentemente usar APIs para estender a funcionalidade de seus programas, em vez de para escrever o mesmo programa de forma mais sucinta (como com hofs). Documentação e instrução são, no entanto, ferramentas instrucionais essenciais. Em ambos os contextos, fazendo descobertas sobre como os programadores aprendem com Documentação da API potencialmente relevante para o nosso trabalho.

O estudo de Duala-Ekoko e Robillard sobre as perguntas que programadores experientes fazem ao aprender APIs [6] mostra que os programadores frequentemente navegam pela documentação da API com base em tipos e palavras-chave (que se assemelham aos rótulos que pedimos aos alunos para criarem). A teoria de Thayer et al. sobre conhecimento robusto de API [25] sintetiza muitos estudos anteriores discutiram a importância da compreensão Padrões de uso da API, que incluem exemplos de entrada/saída. Na API. Em estudos, os desenvolvedores normalmente têm acesso à documentação que Contém tipos, exemplos e descrições comportamentais. Nós fornecemos. documentação somente após as duas primeiras etapas do nosso estudo, e até mesmo então estava em uma forma modificada de uma tabela que resumia os tipos e características (fig. 1), em vez das descrições comportamentais mais ricas encontradas na documentação da API. Isso foi intencional em nosso estudo, para que pudéssemos explorar como o uso prévio do código subjacente a um hof pelos alunos influencia o uso do código pelos alunos. levaria à compreensão de exemplos de entrada/saída.

4 UM PEQUENO GLOSSÁRIO DE ORDEM SUPERIOR

FUNÇÕES

Presumimos que o leitor tenha um conhecimento básico da norma. hofs. No entanto, a figura 1 fornece um resumo dos principais hofs que... uso neste artigo (e suas características distintivas).

Devido ao ponto do currículo em que realizamos estas atividades, Em nossos estudos, focamos exclusivamente em funções em vez de listas. O mapa aplica sua A função `filter` aplica seu argumento a cada elemento de uma lista. A função recebe um argumento booleano, que deve retornar um valor booleano, para cada elemento de uma lista, e usa esse valor booleano para decidir se deve ou não manter o elemento. ou remova esse elemento do resultado. O take-while é sutilmente diferente. Do filtro: ele retém o prefixo para o qual o valor booleano é verdadeiro.

e, ao encontrar o primeiro valor falso, descarta o sufixo. ormap também aceita um Função geradora booleana, que se aplica a cada elemento, e calcula a disjunção dos valores booleanos produzidos; ele interrompe a operação assim que obtém o primeiro valor não falso. `fold` é uma generalização. ciclo de acumulação.

Por que essas funções? Escolhemos map, filter e fold porque São três hofs comuns e canônicos. Escolhemos take-while para sua semelhança com o filtro. (Com estado, o take-while pode ser expresso) (Usando filtro, mas estamos operando em um ambiente puramente funcional.) Escolhemos o ormap porque ele produz resultados que não são listas e porque Os alunos tinham visto isso no texto, mas não precisaram usar, ao contrário de sua aplicação. irmão e mapa, exigindo alguma transferência.

Antes do nosso estudo, os alunos tinham visto todas essas funções, exceto demorado. No entanto, eles não haviam sido expostos a um sistema uma forma de pensar sobre as características comportamentais dessas funções, como dadas pelas colunas na figura. Essas funções têm ligeiramente nomes e ordens de argumentos diferentes em idiomas individuais. Em Em nosso estudo, permitimos que os alunos escrevessem textos livres que nós interpretado, permitindo que os alunos usem quaisquer nomes razoáveis para essas funções.

5 CONTEXTO PEDAGÓGICO

Este trabalho se situa em uma universidade privada altamente competitiva. (ensino superior) nos EUA. O contexto é um curso introdutório acelerado. Cerca de dois terços eram alunos do primeiro ano (normalmente 18 anos de idade); os restantes já tinham cursado pelo menos um semestre de ensino superior. Cerca de 10% não tinham qualquer experiência prévia em informática, e os restantes... tendo cursado algumas disciplinas de informática (no ensino médio), tanto quanto... Curso AP CS A, com alguns alunos tendo ido além. Quase todos eram novatos em programação funcional.

Para ingressar no curso acelerado, todos os alunos tiveram que concluir Um módulo de um mês (durante o verão de 2020) que ensina o básico. programação funcional usando Como Projetar Programas [8], com avaliação aproximadamente a cada dez dias. Esse material aproximadamente compara-se ao primeiro mês de um curso introdutório com ritmo convencional. curso na mesma universidade. Ao final do módulo, os alunos foram apresentados aos hofs na Parte Três (https://httpd.org/2019-02-24/part_three.html) do livro, que os ensina como abstrações

sobre o código. A quarta e última avaliação pedia aos alunos que refizessem Os mesmos problemas da terceira avaliação, mas usando apenas funções de ordem superior e sem recursão explícita. Esta tarefa exigia que os alunos trabalhassem com as funções ``map``, ``filter``, ``fold`` (como ``foldr``, não ``foldl``) e ``andmap`` (esta última é análoga a ``ormap``, como mostrado na tabela), frequentemente em composição. Havia 5 problemas no total, incluindo remover as vogais de uma lista de palavras (``filter`` e ``map``), produzir o subconjunto de palavras cujos caracteres estavam todos em uma lista de referência (``filter`` e ``andmap``) e produzir uma lista de elementos únicos a partir de uma lista de entrada (``fold``, com ou sem ``filter``).

As quatro primeiras etapas do nosso estudo ocorreram entre o final do processo de seleção (agosto de 2020) e o início do curso acelerado (setembro de 2020), após os alunos receberem a decisão de admissão no curso. A quinta e última etapa do estudo relatada neste artigo ocorreu ao final do semestre.

Durante o semestre (outono de 2020), os alunos foram incentivados, mas não obrigados, a usar os hofs. Embora tenham visto os hofs sendo usados repetidamente, não receberam nenhuma instrução adicional sobre eles, presumindo-se que a preparação de verão (incluindo as quatro primeiras etapas) fosse suficiente.

O processo de colocação profissional e os dois primeiros terços do curso propriamente dito utilizaram programação funcional pura, com a introdução e o uso extensivo de estados no último terço do curso. O processo de colocação profissional foi realizado inteiramente em Racket, enquanto o semestre foi inteiramente em Pyret.

6 INSTRUMENTOS E MÉTODOS

Nossos estudos utilizam duas representações diferentes de exemplos de comportamento hof. A primeira são exemplos de entrada/saída, como 1 (listar "vermelho" "verde" "azul").
____»
(lista 3 5 4)

Nossa lista completa de exemplos aparece na figura 2. Seleccionamos cuidadosamente os exemplos para explorar os recursos nas colunas da figura 1. Esses mesmos exemplos foram usados em todas as etapas do nosso estudo que utilizaram exemplos.² No exemplo acima, uma lista de três strings é transformada em uma lista de três números. Esses tipos limitam bastante o conjunto de funções candidatas. A Figura 1 mostra o porquê: ``filter`` e ``take-while`` produzem listas do mesmo tipo que consomem, portanto, não podem ter esse resultado; ``ormap`` não produz uma lista. Em contraste, ``map`` é uma forte candidata: ela permite transformar o tipo da entrada para a saída, mas, ao fazer isso, deve preservar o número de elementos; cada elemento de saída deve ser gerado de forma consistente a partir da entrada correspondente. É fácil ver que cada número de saída tem o comprimento da string de entrada correspondente. Este é, de fato, o tipo de exemplo de ``map`` comumente encontrado em livros e tutoriais.

O leitor atento notará que o operador `fold` também poderia ter gerado essa saída. De fato, o operador `fold` possui uma propriedade de universalidade (uma breve exposição é dada por Hutton [18]) que o torna capaz de expressar todos os outros operadores que estamos estudando neste artigo. Em outras palavras, todo exemplo de entrada/saída poderia (também) ser gerado pelo operador `fold`. Relatamos como levamos isso em consideração ao apresentar os resultados de cada etapa do estudo na seção 7.

1Os exemplos são escritos na sintaxe do Racket [11], seguindo How to Design Programs [8], mas como envolvem apenas dados e nenhum código, são extremamente fáceis de traduzir para qualquer número de outras linguagens de programação.
2Exceto por duas pequenas edições feitas após a Etapa 1 para reduzir a confusão: o último elemento da entrada do exemplo 11 mudou de 1 para 5, e a entrada do exemplo 16 passou de 1 7 3 -1 -4 2 8 9 -5 para 1 7 2 3 -1 -4 2 7 8 9 -5. A Figura 2 incorpora essas duas edições.

Nossa etiqueta de referência para cada exemplo está marcada na figura 2 (estas não foram mostradas aos alunos). Nossas etiquetas reservam a dobra para exemplos que não podem ser produzidos por um hof mais especializado.

O segundo instrumento mostrou o comportamento por meio de diagramas, que são apresentados na figura 3. Os diagramas usam espaço e cor para capturar características principais da maioria dos hofs da figura 1. (A) pretende mostrar um mapa; (B) um filtro; (C) é um distrator intencionalmente ambíguo (poderia ser interpretado como um filtro, embora a ausência de uma linha até o final possa sugerir uma computação interrompida, como `take-while`, exceto que o prefixo inteiro não está incluído); e (D) é (um caso de) `fold`, embora também possa ser versões especializadas como `ormap`.

Os alunos não tinham sido apresentados a essas imagens anteriormente, nem lhes foram explicadas formalmente.

Tarefas de estudo. Os alunos utilizaram essas representações em dois tipos de tarefas: as tarefas de agrupamento solicitavam que os alunos formassem grupos com os exemplos textuais (fig. 2) e sugerissem nomes para esses grupos. As tarefas de classificação solicitavam que os alunos rotulassem os diagramas e os exemplos textuais com descrições ou nomes de funções cognitivas que poderiam produzir o comportamento representado.³ A Seção 7 apresenta as motivações, instruções e resultados para cinco etapas de estudo utilizando essas tarefas.

Métodos. A coleta de dados para cada etapa ocorreu por meio do Google Forms, com a coleta de identidade ativada em quatro das cinco etapas (acidentalmente, esquecemos de ativar a coleta de identidade na quarta etapa). Os alunos receberam instruções por escrito no fórum de discussão do curso, incluindo um link para participar do estudo. Os alunos foram incentivados a realizar todas as etapas (motivados como tarefas que os ajudariam a se preparar para o curso), mas, devido às diversas pressões criadas pela COVID-19, todas foram tornadas opcionais. A quinta etapa (também opcional) foi realizada logo após a última tarefa do semestre, três meses após o término da quarta etapa do estudo.

Fundamentalmente, nós dois autores (um dos quais era o professor da disciplina; o outro não tinha vínculo com o curso) concordamos sobre como classificar cada exemplo. Como havia apenas 21 exemplos, não havia itens suficientes para usar um processo como a confiabilidade interavaliadores tradicional. Em vez disso, simplesmente discutimos cada exemplo até chegarmos a um consenso completo. Os exemplos foram ordenados aleatoriamente (como mostrado na figura) para que exemplos de funções individuais não fossem agrupados; essa mesma ordem foi usada em todas as etapas. Depois de concordarmos com os rótulos, um dos autores realizou as análises e fez a codificação manual.

População. Um total de 114 alunos concluíram o curso. Cerca de 20% eram do sexo feminino. Apenas alguns eram negros ou hispânicos/latinos. O número de alunos que responderam a cada etapa foi o seguinte: N1 = 68, N2 = 70, N3 = 76, N4 = 64 e N5 = 83. Os alunos que responderam não são todos iguais e representam cerca de metade a dois terços da turma. Pelo menos 28 alunos realizaram todas as etapas 1, 2 e 5, e pelo menos 38 realizaram as etapas 1 e 5 (uma contagem exata é difícil porque alguns alunos trocaram seus endereços de e-mail entre as rodadas).

Não podem ser mapeados de forma confiável, portanto, esses números representam uma ligeira subestimação.

Nossa terminologia é diretamente inspirada nos termos correspondentes em ciência de dados e aprendizado de máquina: "clustering" agrupa objetos semelhantes (sem necessariamente atribuir-lhes um rótulo semanticamente significativo), enquanto "classification" aplica rótulos semânticos aos objetos.

1 FILTRO/TAKE-WHILE (lista "cs019" "ma054" "cs033" "cs018" "visa039") —» (lista "cs019")	12 TAKE-WHILE (lista verdadeiro verdadeiro falso verdadeiro falso verdadeiro falso) —» (lista verdadeira verdadeira)
2 MAPA (lista 2 1 3) —» (lista (lista "a" "a") (lista "a") (lista "a" "a" "a"))	13 FILTRO (lista 1 4 4 2 6 1) —» (lista 1 1)
3 QUALQUER COISA (lista 1 2 3 4) —» (lista 1 2 3 4)	14 DOBRAS (lista "cs019" "ma054" "cs033" "cs018" "visa039") —» 2
4 FILTROS (lista "cs019" "ma054" "cs033" "cs018" "visa039") —» (lista "ma054" "visto039"))	15 DOBRAS (lista 1 2 3 4 5) —» (lista 1 4 9 25)
5 FILTROS (lista (lista "a") (lista "b") (lista "d") (lista "e")) —» (lista (lista "a") (lista "e"))	16 LEVE UM TEMPO (lista 1 7 2 3 -1 -4 2 7 8 9 -5) —» (lista 1 7 2 3)
6 Qualquer coisa menos MAPA (lista 4 6 2 1) —» vazio	17 FILTRAR/LEVAR POR UM TEMPO (lista (lista 2 3) (lista 1) (lista 4 5 2) (lista) (lista 2 7)) —» (lista (lista 2 3) (lista 1) (lista 4 5 2))
7 DOBRAS (mas poderia ser um MAPA que aceita sufixos de lista) (lista 1 2 3 4) —» (lista (lista 10 6 3 1) (lista 6 3 1) (lista 3 1) (lista 1))	18 DOBRAS (ou MAPA: lam(x): 4 - x fim) (lista 1 2 3) —» (lista 3 2 1)
8 MAPA (lista "vermelho" "verde" "azul") —» (lista 3 5 4)	19 MAPA (lista add1 sub1) —» (lista (lista 2 3 4) (lista 0 1 2))
9 ORMAP (lista verdadeiro verdadeiro falso verdadeiro falso verdadeiro falso) —» verdadeiro	20 DOBRAS (lista 1 2 3 4) —» 24
MÉDIA DE 10 VEZES (lista 1 4 4 2 6 1) —» 3	21 FILTRO/TAKE-WHILE (lista 1 7 3 -1 -4) —» (lista 1 7 3)
11 MAPA (lista 4 6 2 5) —» (lista 1) 1 1 1)	

Figura 2: Exemplos de entrada/saída utilizados no estudo, anotados com seus respectivos rótulos de verdade fundamental.

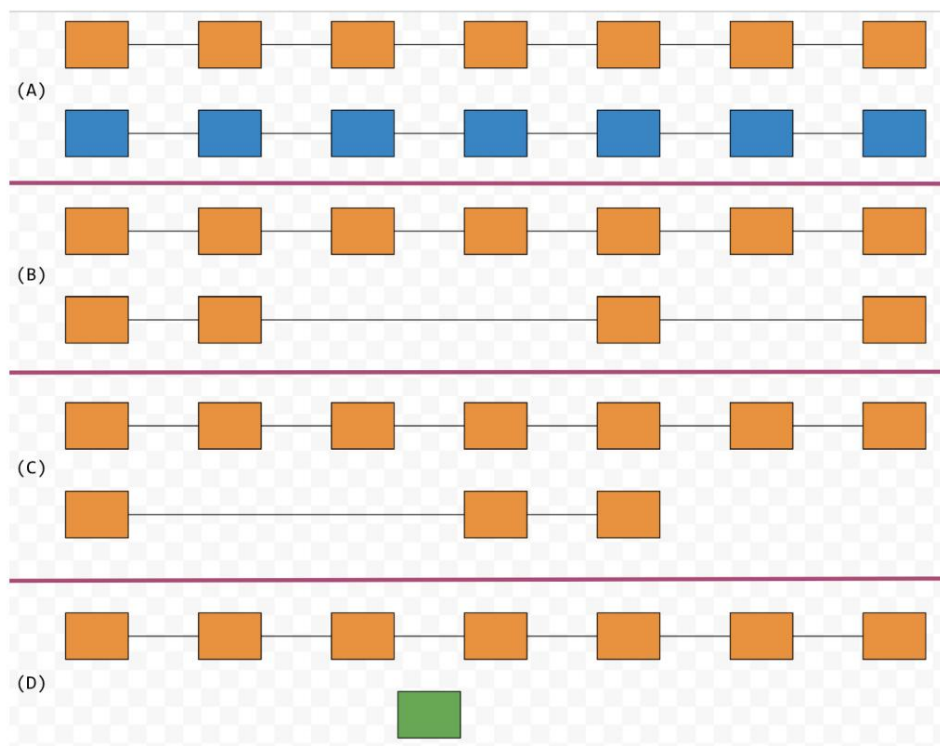


Figura 3: Imagens para rotular (a maioria das caixas é laranja, exceto a segunda linha de (A) é azul e a segunda linha de (D) é verde)

7. DESCRIÇÃO E ANÁLISE DO ESTUDO

7.1 Etapas 1 e 2: Agrupamento

Nas etapas 1 e 2, os alunos receberam a lista completa de exemplos (fig. 2) (sem os rótulos de Hof) e foram solicitados a construir agrupamentos. As instruções específicas foram as seguintes:

Abaixo, apresentamos uma lista de 21 pares de entrada/saída. Sua tarefa é agrupá-los de acordo com a sua percepção de similaridade.

O que é "agrupamento"? Significa formar grupos de coisas onde todos os elementos de um grupo são semelhantes entre si, mas diferentes dos elementos dos outros grupos.

Quando dois pares devem ser agrupados no mesmo cluster? Quando os dois pares poderiam ter sido processados pela mesma função de ordem superior (possivelmente incluindo funções que você ainda não viu).

Portanto, é preciso considerar as relações entre a entrada e a saída em cada par (tipo, ordem, etc.). Dentro de cada grupo, todos os exemplos devem compartilhar essas relações.

Pediremos que você descreva essas relações depois de formar seus grupos. Identificar essas relações é o ponto central deste exercício.

Você pode fazer quantos grupos quiser (até 8).

Você pode encontrar pares que se encaixam em vários grupos. Se isso acontecer, crie um grupo separado para todos eles e explique por que você acha que são ambíguos.

Os alunos também foram solicitados a dar nomes significativos aos clusters e foram especificamente instruídos: "Observe que um bom nome para um cluster pode ser uma função de ordem superior que realizaria esse mapeamento (quando fornecida uma função adequada como parâmetro)."

7.1.1 Análise. Criamos agrupamentos de referência com base nos rótulos dos exemplos na figura 2. Isso resulta em seis agrupamentos (map, filter, fold, ormap, takewhile, anything). Reservamos o agrupamento fold para exemplos que não podem ser produzidos por um agrupamento hierárquico mais especializado. Comparamos os agrupamentos de cada aluno com os nossos agrupamentos de referência. Como se tratam de comparações de conjuntos, utilizamos o índice de Jaccard, $\hat{y}$ que atribui uma pontuação entre 0 (nada em comum) e 1 (os conjuntos são idênticos). Nossa função de pontuação exata é mostrada na figura 4.

Com 6 agrupamentos de referência, uma pontuação perfeita seria 1 por agrupamento, totalizando 6. A pontuação mínima não é exatamente 0: como cada exemplo é forçado a ser alocado em algum agrupamento, cada agrupamento de aluno corresponderá a algum agrupamento de referência. De fato, nossa menor pontuação observada foi 0,33, para um aluno que colocou todos os exemplos em um único agrupamento (observando que todos poderiam ser implementados por meio da dobra universal: seção 6).

Observe que nossa função de pontuação pode gerar uma superestimação generosa! Para um determinado aluno, vários de seus agrupamentos podem estar próximos de um único agrupamento verdadeiro. Cada um deles receberia uma pontuação bastante alta. Em vez disso, para cada grupo de alunos, devemos remover o grupo de referência que apresenta a pontuação mais alta e prosseguir apenas com os grupos restantes. (Técnicamente, esse algoritmo guloso pode não ser adequado para todos os grupos.)

40 índice de Jaccard (ou similaridade de Jaccard) de dois conjuntos, A e B , é $|A \cap B| / |A \cup B|$.

Para cada aluno:

Para cada grupo de alunos:

Para cada grupo de referência: calculou-se o

índice de Jaccard entre esse aluno e o grupo de referência.

Atribua ao grupo de alunos o seu índice de Jaccard mais alto entre todos os grupos de referência.

Some as pontuações de todos os grupos de alunos.

Figura 4: Nosso algoritmo para pontuação de clusters

produzir a pontuação mais alta possível; em vez disso, temos que considerar todos esses pares de clusters e encontrar a pontuação somada máxima.) Optamos por não ajustar para essa superpontuação em nossa análise.

7.1.2 Resultados. Após a Etapa 1, encontramos uma média de $\mu = 2,51$ (com $\sigma = 0,66$, $\bar{y} = 0,81$) e mediana de 2,52. Esses resultados sugerem que os alunos têm alguma capacidade de realizar essa classificação, mesmo no início do uso de funções de saída de Hof. Em particular, muitos alunos foram capazes de reconhecer comportamentos básicos semelhantes a mapas e filtros. No entanto, vários aspectos se destacaram em suas respostas escritas: • Uma falha em

compreender que múltiplos operadores podem produzir os mesmos resultados.

Acreditamos que haja um certo grau de comportamento semelhante à fixação funcional [7] nessas respostas: se um operador produz um determinado resultado, outro o produzirá. não.

- Dificuldade com a função 'fold', que de qualquer forma é uma das funções de agrupamento mais complexas (já que simula um laço de acumulação).
- Muitos casos em que não conseguiram usar o nome de um operador, mesmo tendo fornecido uma descrição significativa: por exemplo, "Aplicação de função que mantém a ordem, membro por membro" em vez de "map".
- Várias respostas focaram nos tipos, mas não nas operações: por exemplo, "Lista de inteiros não aninhada" ou "retorna uma lista de listas" como rótulo de um cluster.

Observando a ordem em que os alunos apresentaram seus clusters, os primeiros tendiam a ter definições muito mais concisas e correspondiam com muito mais frequência a nomes de operadores.

Os agrupamentos posteriores eram muito mais restritos e até incluíam palpites explícitos (por exemplo, "Inverso?").

No contexto de nossas questões de pesquisa, observamos que muitos alunos reconheceram características (como ordem, comprimento da saída e tipo de retorno) e conectaram alguns comportamentos de HFOs aos seus nomes. Os alunos tiveram mais dificuldade em descrever seus agrupamentos quando não conseguiram conectá-los aos nomes dos HFOs. De modo geral, as lacunas entre nossa verdade fundamental e seus agrupamentos sugeriram que vários alunos ainda estavam longe de dominar como os exemplos se alinhavam a HFOs específicos.

Após a Etapa 1, notamos dois problemas em nossos exemplos que podem ter levado os alunos a se concentrarem em propriedades excessivamente específicas (ver nota de rodapé na seção 6). Após fazermos pequenos ajustes para evitá-los e aprimorarmos as instruções de agrupamento para evitar algumas confusões que observamos, pedimos aos alunos que tentassem novamente (sem feedback ou instruções adicionais). Observe que esses dois fatores, combinados com a familiaridade com os exemplos, podem, por si só, levar os alunos a um desempenho um pouco melhor. Usando o mesmo esquema de pontuação, desta vez os resultados dos alunos produziram uma média $\mu = 2,64$ (com $\sigma = 0,70$, $\bar{y} = 0,83$) e mediana $^2 = 2,51$. Isso parece semelhante aos resultados da Etapa 1. Constatamos que 43 alunos enviaram respostas para ambas as etapas, então realizamos uma comparação intraindividual usando

Foi realizado um teste t de amostras pareadas para médias. Para um nível de significância de $\bar{y} = 0,05$, obtivemos um valor p de 0,21, indicando que não houve mudança significativa para esses alunos.

Apesar disso, observamos melhorias notáveis na prosa, talvez devido à prática e ao aprimoramento das nossas instruções. Ainda assim, constatamos alguns dos fenômenos descritos anteriormente (por exemplo, o uso do rótulo "Manter Elementos Específicos" em vez de "filtrar"). Diante da persistência dessas características, decidimos tentar uma estratégia alternativa.

7.2 Etapa 3: Rotulagem

Após os dois exercícios de agrupamento, explicamos a intenção dos exercícios aos alunos e fornecemos (uma versão do) gráfico da figura 1. Também decidimos que a tarefa de classificação era muito onerosa e ambígua (devido à falta de uma separação estrita dos elementos — tanto porque alguns exemplos eram intencionalmente ambíguos, quanto devido à universalidade da dobra). Decidimos tentar novamente com uma atividade mais leve (e potencialmente menos ambígua).

A nova atividade apresentava os diagramas da figura 3 e pedia aos alunos que os identificassem. As instruções eram as seguintes:

Para as quatro imagens abaixo, dê a cada uma delas um rótulo o mais significativo e conciso possível e explique por que você a escolheu. Pode ser útil pensar em termos das características que você usou no teste anterior.

Juntamente com essas instruções, o instrutor disse aos alunos: "Depois que vocês terminarem, publicarei minhas considerações sobre esses exercícios, para que vocês tenham o conteúdo fresco na memória a tempo do início da aula." (Lembrem-se de que esses exercícios ocorreram antes do início do semestre, mas depois que os alunos concluíram o processo de nivelamento para serem admitidos no curso.)

7.2.1 Análise. Como cada figura foi desenhada com um rótulo de função específico em mente (exceto no caso do distrator), pudemos simplesmente comparar os rótulos dos alunos com nossa intenção. Um dos autores realizou a avaliação.

7.2.2 Resultados. Como os diagramas destacavam diretamente certas características dos hofs — notadamente o tipo de elemento, o tipo de saída e o comprimento da saída — esperávamos que os alunos tivessem mais facilidade em associar os diagramas a hofs específicos. Nossos resultados foram um tanto surpreendentes.

Para (A), mapa (o hof pretendido) foi de fato a resposta mais comum (pouco mais da metade usou a palavra "mapa" de alguma forma razoável, principalmente apenas "mapa"). Alunos usaram o que são efetivamente

⁵ Em retrospectiva, percebemos que nossas figuras provavelmente eram abstratas demais. Como uma possibilidade a ser considerada, Rodrigo Duran sugeriu o uso de formas além das cores, para reduzir o salto de abstração.

Alguns sinônimos incluíam “transformar tudo”, “aplicação de função elemento a elemento” ou “percorrer todos os elementos com base em uma única entrada”. Outros eram menos claros, como “modifica todos os elementos” ou “todos os elementos processados para obter os produtos na lista”. Alguns eram ambíguos, misturando descrições de função e visuais: “retângulos laranja mapeados um a um para azul” ou “Mapear a “lista” laranja para a “lista” azul”. Por fim, vários se concentravam puramente no visual: como “mudar cor”, “trocar cores” ou “converter tudo para azul”. 10% focavam exclusivamente nas cores.

(B) foi concebido para ser um filtro, e as respostas seguiram o mesmo padrão:

- Mais da metade usou a palavra “filtrar” explicitamente de alguma forma.
- Alguns usaram sinônimos, como “Selecionar”.
- Alguns usaram frases vagamente semelhantes, mas insuficientemente descritivas: “Retornando um subconjunto de elementos em uma lista de comprimento original”, “Substituir”, “Buscar e Retornar”.
- Cerca de 10% focaram em atributos físicos: por exemplo, “Remover as caixas 3, 4 e 6”, “Grupos de caixas têm o mesmo comprimento total de caixas conectadas, mas o número de caixas é diferente para cada grupo”, “desanexar”.
- Alguns combinaram descrições abstratas e físicas: “Selecionando alguns/filtrando 7 retângulos laranja”, “Filtrar lista de laranjas para lista de laranjas”.

Lembre-se de que (C) era uma alternativa de distração. As respostas, no entanto, sugerem que os alunos captaram bem o estímulo visual:

- Cerca de um terço dos alunos classificou-o como alguma versão de filtro.
- Cerca de um quinto interpretou a frase curta como truncamento, criando uma descrição de um filtro abreviado: por exemplo, “Filtrar com Saída”, “filtrar até”, “Selecionar e truncar”, “tipo filtro”, “Filtrar e Cortar”, “Fatiar e filtrar”, “Filtro Limitado?”, “filtro parcial”.
- Talvez surpreendentemente, muito poucos alunos fizeram interpretações físicas: “Selecionando alguns / filtrando 7 retângulos laranja”, “filtrar_cinco”, “desagrupar e encurtar”.
- Cerca de 10% das respostas foram difíceis de entender e classificadas. sify (por exemplo, “filtrar 2x”, “Selecionar os primeiros”, “Número diferente de caixas e comprimento total do grupo de caixas”), talvez refletindo a natureza distratora do problema.

Por fim, (D) tinha como objetivo representar uma função fold ou casos especiais dela que não produzem listas, como ormap. Metade dos alunos escolheu nomes precisos como fold, reduce e ormap. No entanto, muitos alunos se confundiram com o valor único de uma cor diferente e se concentraram nessa característica: houve várias respostas como “retorna um novo valor”, “manipulaTodasAsCaixasParaUma”, “colapsa”. Alguns não conseguiram expressar nenhuma característica útil de alto nível, como “gera um resultado com base nas entradas” (o que descreve todas as funções!).

Algumas afirmações eram possivelmente incorretas, como “Constante, tipo diferente” ou “filtrado por contagem”. Por fim, observamos novamente que cerca de 8% dos alunos combinaram forma e função: por exemplo, “Dobrar a lista laranja para verde”, “Dobrar 7 retângulos para qualquer cor”, “Reduzir e mudar de cor”, “Condensar em uma caixa verde”.

De modo geral, ficamos surpresos ao ver que mais de 10% dos alunos se concentraram, pelo menos parcialmente, na notação visual (incluindo a cor) em vez do comportamento representado. Isso sugere que os alunos não compreenderam realmente o objetivo da atividade, apesar de essa explicação ter sido feita na publicação que disponibilizou o exercício.

Notamos que muitos dos alunos que se concentraram exclusivamente no aspecto visual não haviam concluído as etapas anteriores, portanto, eles podem não...

compreenderam totalmente a tarefa e podem não ter prestado atenção às instruções. Em contrapartida, a maioria dos alunos que deram respostas híbridas havia concluído uma ou ambas as etapas anteriores. No entanto, nossa conclusão geral foi que a versão baseada em diagramas talvez fosse muito aberta à interpretação. Diante disso, não fizemos comparações aprofundadas dos resultados dos alunos que concluíram tanto a Etapa 3 quanto uma das duas primeiras etapas (pelo menos 56 alunos).

Em relação às nossas questões de pesquisa, constatamos que aproximadamente metade dos alunos associa corretamente os padrões aos nomes dos hof (RQ 2). Os demais estão detectando alguns padrões, mas nem todos são comportamentais e não estão necessariamente alinhados com as funções que os alunos aprenderam durante a pedagogia inicial (RQ 1). Com base nas respostas que enfatizaram os recursos visuais dos diagramas, decidimos tentar novamente a abordagem orientada à rotulação, mas desta vez com os exemplos textuais.

7.3 Etapas 4 e 5: Classificação Contrariando a

promessa anterior de que a Etapa 3 seria a última, o instrutor publicou uma “última etapa”, acompanhada de uma longa explicação. (O espaço não permite incluir o texto completo.) Mencionou brevemente o objetivo educacional e, em seguida, descreveu como o instrutor pensa sobre as funções em termos de suas características. A publicação compartilhou a tabela mostrada na figura 1. Finalmente, os alunos foram informados: “Com base em tudo isso, gostaria de solicitar que vocês preencham um questionário final, que deverá concluir o treinamento sobre este tópico.” As instruções específicas eram as seguintes:

Para cada par, selecione todas as funções de ordem superior que poderiam (razoavelmente) ter transformado a entrada na saída correspondente.

Os alunos foram solicitados a marcar uma ou mais caixas com as etiquetas “mapa”, “filtro”, “tempo”, “dobrar” e “Outro” (que permitia a inserção de texto livre). Optamos por não incluir “mapa” para que os alunos tivessem que digitá-lo usando a opção “Outro”.

7.3.1 Análise. Comparamos as respostas dos alunos com as nossas respostas de referência (da figura 2). Para todos os exemplos, aceitamos respostas com a operação de “fold” : por exemplo, uma resposta com “filter” e “fold” foi colocada no mesmo grupo que uma resposta apenas com “filter” . Assim, os alunos nunca foram penalizados por usar a operação mais geral além da mais específica. Na discussão a seguir, abordaremos a operação de “fold” mais especificamente para exemplos particulares e como um fenômeno geral.

7.3.2 Resultados antes do início do curso (Etapa 4). O gráfico à esquerda na figura 5 exibe a frequência de erros (eixo y) como uma porcentagem das respostas, agrupadas a cada 5% (eixo x). Para facilitar a leitura do gráfico: ele indica que 6 (eixo y) dos 21 exemplos têm menos de 5% de respostas incorretas, mas um exemplo apresenta 70-75% (especificamente, 73,4%) de respostas incorretas. Como os exemplos são muito diferentes entre si, não acreditamos que estatísticas resumidas sobre esses erros sejam significativas.

Usando a notação En para representar o exemplo n da figura 2, as baixas taxas de erro estão em map (E2 (1,6%), E8 (0%), E11 (3,1%), E19 (12,5%)), fold (E10 (0%), E14 (4,7%), E20 (0%)) e map/fold (E18 (12,5%)). Organizamos os valores em ordem de exemplo, em vez de porcentagem, para facilitar a consulta na figura 2.

Os erros são um pouco maiores nos exemplos com apenas filtro (E4 (15,6%), E5 (17,2%), E13 (14,1%)) por um motivo simples: alunos

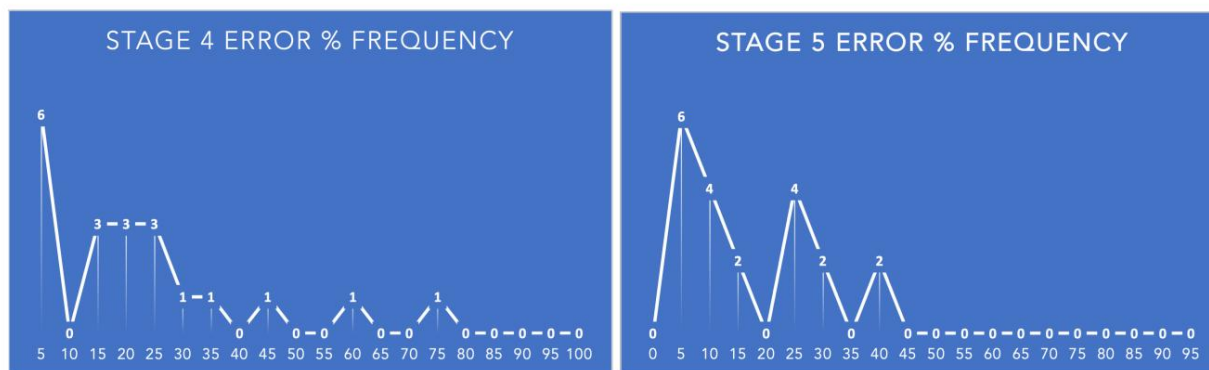


Figura 5: Frequências de erro nos estágios 4 (esquerda) e 5 (direita)

Costuma-se também marcar esses exemplos como adequados para "levar um tempo"! Todos os erros nesses exemplos são causados por essa confusão.

Nos dois exemplos que continham apenas a opção "take-while", as taxas de erro foram um pouco maiores: E12 (25,0%) e E16 (26,6%). Todos os erros decorrem de alunos que também marcaram incorretamente a opção "filter". Em um exemplo, no exemplo E12, um aluno marcou apenas a opção "fold". Os números gerais sugerem um possível viés, no qual os alunos são mais propensos a classificar erroneamente a opção "take-while" como "filter" do que o contrário. Não temos uma explicação para esse fenômeno (se for verdadeiro), embora uma hipótese seja a maior familiaridade dos alunos com a opção "filter" nesse ponto.

Em dois exemplos que poderiam ser tanto de filtro quanto de take-while, as taxas de erro não foram muito altas: E1 (18,8%), E21 (17,2%). Em ambos os casos, os únicos erros ocorreram quando os alunos se esqueceram de uma ou outra possibilidade. No entanto, o viés não é uniforme. No primeiro exemplo (E1), 14,2% dos alunos (não entre os que cometeram erros!) negligenciaram a possibilidade de take-while.⁷ No segundo exemplo (E21), 15,9% ignoraram a possibilidade de filtro. No terceiro exemplo, a taxa de erro é quase o dobro: E17 (34,4%).

Aqui, 3% ignoraram o comando "take-while", mas dez vezes mais ignoraram o comando "filter", talvez porque a lista vazia pareça um elemento sentinela. De qualquer forma, é notável que, em todos esses exemplos, os alunos nunca confundiram nenhum dos dois com o comando "map", apenas entre si. Isso sugere que os alunos observaram com precisão a característica de ordem superior (hof) em relação à presença de todos os itens ou de um subconjunto na saída, mas talvez não estejam prestando atenção a quais elementos estão sendo descartados (um sufixo, como em "take-while", versus elementos intercalados, como em "filter").

Três erros graves surgem nos cinco exemplos com alta incidência de erros. • Falha em marcar todos os hofs apropriados: Tanto em E3 (56,3%) quanto em E6 (23,4%), múltiplos hofs além de fold foram aplicados. Em E3, 60% dos erros ocorreram devido à omissão de fold, enquanto os outros 40% omitiram outra função aplicável. Em E6, todas as opções, exceto map, poderiam ter sido aplicadas e, de fato, ninguém a selecionou (as omissões ocorreram entre as outras opções). Isso sugere, mais uma vez, uma compreensão sólida do recurso de hof em relação a

Não sei se as listas de saída e as listas de entrada têm o mesmo comprimento, mas não entendo bem o conceito de fold.

- Marcação de funções de busca cujos tipos não correspondem aos exemplos: No E9 (23,4%), a maioria das respostas errôneas escolheu apenas "filter" e/ou "take-while", nenhuma das quais está correta em termos de tipo (pois um único booleano, e não uma lista, foi retornado). Alguns escolheram uma ou ambas, além de "ormap", que está correto em termos de tipo. Isso sugere que os alunos estavam prestando mais atenção ao comprimento relativo da entrada do que ao tipo.
- Manipulação incorreta de detalhes posteriores nas listas: O E15 (40,6%) foi intencionalmente sutil e escolhido como um teste de atenção: à primeira vista, parece um mapa (de uma função de quadratura), mas a entrada tem cinco elementos, enquanto a saída tem apenas quatro. Muitos alunos usaram a opção "outro" para escrever uma declaração, por exemplo, perguntando se o problema estava incorreto ou indicando que, se um 16 estivesse presente, eles o classificariam como um mapa (classificamos essas respostas como corretas).

Ao longo dos nossos dados, observamos evidências de que os alunos têm dificuldades com o "fold" (o hof de propósito geral). Como já mencionamos, o "fold" é o hof mais difícil de usar, presumivelmente porque suporta todas as funcionalidades principais (transformar, ignorar e agregar elementos). De fato, uma maneira de entender o "fold" é através desse suporte a múltiplas funcionalidades (embora essa perspectiva seja difícil de detectar com base no código ou nos tipos não abstraídos). Os dados para o exemplo E15 (40,6%) mostraram que os alunos não compreenderam totalmente essa perspectiva: 13,3% dos alunos escolheram tanto "map" quanto um dos seguintes: "filter" ou "take-while", opções que reconhecem que o exemplo apresenta tanto transformação quanto elementos ignorados. Idealmente, os alunos entenderiam que tais situações exigem o uso do "fold" (ou uma composição de hofs, como 23% dos alunos fizeram).

E7 (73,4%) é um exemplo um pouco complexo que também levanta questões sobre quais características os alunos podem estar percebendo (mesmo os autores inicialmente o classificaram erroneamente, considerando-o um problema de dobra, mas sem perceber que também pode ser um mapa). O exemplo preserva o comprimento da lista (sugerindo um mapa), mas a transformação dos elementos não é imediatamente óbvia (sugerindo uma dobra). Um aluno sugeriu (de forma livre) uma função para construção de listas, "build-list", que é aplicável de forma ampla, visto que é frequentemente usada para gerar sequências de

⁶ É possível que a ideia de usar filter e take-while em listas de booleanos (o que é bastante raro na prática e não havia sido feito nos trabalhos que eles tinham visto até então) fosse suficientemente desconhecida para eles, a ponto de não reconhecerem essas possibilidades.

⁷ Em retrospectiva, este exemplo foi mal concebido: como o nome do curso é "cs19", os alunos podem ter instintivamente esperado que se tratava de um filtro para o curso em questão. Só percebemos isso ao escrever este artigo.

Exemplos de rótulos: Taxa de erro na Etapa 4; Melhoria na taxa de erro na Etapa 5.

1	filtro, take-while	22.9	-4.1	18,8		
2			mapa	1.6	1.2	0,4
3			filtro de	56,3	14,5	41,8
4			qualquer coisa	15,6	8.4	7.2
5			filtro	17.2	10.8	6.3
6			Qualquer coisa menos dobradura	23,4	20,5	3.0
7			de mapa	73,4	36.1	37,3
8			mapa	0,0	1.2	-1,2
9			dobra	23,4	21,7	1.8
10			ormap	0,0	2.4	-2,4
11				3.1	1.2	1.9
12			mapear enquanto	25,0	24.1	0,9
13			filtro	14.1	6.0	8.0
14			dobrar	4,7	9,6	-5,0
15			dobrar	40,6	28,9	11.7
16			levar um tempo	26,6	27,7	-1,1
17			filtro, take-while fold, mapa	34,4	39,8	-5,4
18				12,5	6.0	6,5
19				12,5	3.6	8,9
20			dobradura do mapa	0,0	1.2	-1,2
21			filtro, take-while	17.2	21,7	-4,5

Figura 6: Taxas de erro e diferença

números, mas não pode funcionar aqui porque levaria a um tipo de erro (consome apenas um número, além de uma função).

Em geral, esses resultados corroboram a hipótese de que os alunos são geralmente aprendizes de identificar as principais características dos exemplos de entrada/saída e quais habitats estão associados a cada uma dessas características. Onde os alunos cometem erros, eles não se aprofundaram o suficiente nesses assuntos. associações, falhando em perceber inconsistências em tipos ou detalhes em como os hofs individuais poderiam ser compostos em código real.

7.3.3 Resultados Próximos ao Final do Curso (Etapa 5). Nós poderíamos esperar-se que os conceitos dos alunos sobre HAFs evoluam após o uso. eles ao longo do percurso após as quatro primeiras etapas. Consequentemente, Ao final do semestre (três meses após a Etapa 4), repetimos o processo. o estudo da Fase 4. A Figura 5 compara as frequências de erro para o duas etapas. A Figura 6 mostra as mudanças nas taxas de erro em toda a turma.

em cada exemplo entre a quarta e a quinta etapas.

Tanto a figura quanto os gráficos mostram alguns ganhos significativos, juntamente com alguma piora no desempenho. Dito isso, não podemos tirar conclusões precipitadas. muito conhecimento (ou conforto) a partir desses resultados. Infelizmente, nós não podemos fornecer uma comparação por aluno, pois analisamos criticamente. Não foi possível registrar a identidade dos alunos nos dados da Etapa 4. No entanto, Sabemos que a sobreposição entre os Estágios 3 e 5 é de pelo menos 51%. estudantes, então podemos esperar cerca de 40 estudantes em comum entre 4 e 5. No entanto, independentemente do tamanho da sobreposição, dificilmente deveria ocorrer. Seria surpreendente se um semestre de uso (e ensino) disso acontecesse com este...

De forma geral, esse conceito deve melhorar o desempenho dos alunos.

Talvez o mais preocupante seja que alguns dos erros observados anteriormente... persistir. No E1, 16% dos alunos perderam o tempo de reflexão. No E21, 18%. Filtro não detectado. No E17, 36,1% dos filtros não detectados. Essas taxas de erro são, na verdade, mais altas do que no Estágio 4! Alertamos que estas podem ser devido a diferenças na população; além disso, os exemplos podem ter sido

mais fresco na memória dos alunos do 4º ano. Ainda assim, chegando ao fim. Após um semestre de uso frequente, esses números são reveladores. Semelhante Os padrões persistem em outros exemplos.⁸ Essas semelhanças sugerem ou que nossas populações sejam semelhantes ou até mesmo talvez que estas são erros comuns, mesmo entre programadores com alguma experiência. experiência.

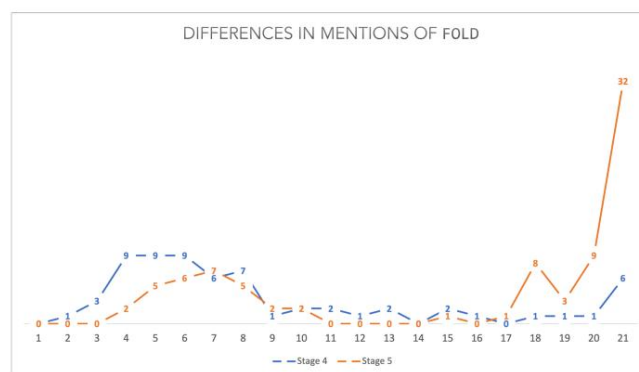


Figura 7: Frequência de dobras nos estágios 4 e 5

A frequência de dobras. Nossa discussão sobre a Etapa 4 apontou para os desafios que os alunos pareciam ter ao usar dobras. Como alunos À medida que adquirimos mais familiaridade com os hofs durante o semestre, poderemos ver.

⁸ Além disso, três alunos escolheram quase todas as funções para todos os exemplos. Não está claro o porquê. se essas escolhas foram feitas intencionalmente ou por alunos que não estavam com vontade de fazê-las exercício. Incluímos esses em nossas taxas de erro, mas eles representam a maior parte dos pequenos declínios e mascaram algumas das melhorias.

Eles reconhecem mais problemas como sendo instâncias de fold. Será que vemos isso em nossos dados?

De fato, observamos isso. Na figura 7, apresentamos graficamente a frequência com que os alunos mencionam a dobra em suas classificações nos Estágios 4 e 5. Parece haver um número significativamente maior de alunos que classificam a maioria ou todos os problemas com a palavra "dobra". As estatísticas descritivas confirmam essa observação: para o Estágio 4, a média é de 8,79 e a mediana de 7; para o Estágio 5, a média é de 15,43 e a mediana de 19. Devido às potenciais diferenças populacionais, não buscamos interpretar essas diferenças mais a fundo, mas simplesmente registramos sua presença e sugerimos que elas não são inconsistentes com o crescimento esperado dos alunos.

7.4 Ameaças à Validade Existem

ameaças naturais à generalização devido ao tamanho da população, localização, especificidades de classe, escolhas de hofs e aos efeitos da COVID-19. Listamos abaixo algumas considerações mais interessantes.

Ameaças à Validade Interna. Diferentes subconjuntos de alunos participaram de cada etapa, o que limita as conclusões que podemos tirar de nossos dados. Dos 114 alunos do curso, um mínimo de 64 e um máximo de 83 participaram das etapas individuais (os dados populacionais na seção 6 sugerem que aproximadamente 35 alunos participaram de todas as etapas). Uma vez que os alunos tomaram conhecimento dos objetivos instrucionais do estudo, os alunos com melhor desempenho podem ter parado de participar, o que poderia ter distorcido os dados. Além disso, a classificação pode ser uma tarefa mais fácil do que o agrupamento, o que poderia explicar por que os alunos parecem ter um desempenho melhor no final. Dito isso, a persistência de certos erros na Etapa 5 indica que métodos mais robustos para o ensino sobre HSFs ainda são necessários.

Validade Ecológica. Em última análise, o que nos interessa é que os alunos apliquem corretamente os algoritmos de agrupamento hierárquico (hofs) durante a programação. Não sabemos se os alunos cometem erros de programação consistentes com as confusões que nossos dados mostram. Por exemplo, será que os alunos usam acidentalmente o operador ``take-while`` quando deveriam usar o operador ``filter``, ou vice-versa? É difícil dizer, pois é possível que os alunos cometam esse erro, mas o corrijam antes de enviar o trabalho — portanto, o custo pode não ser a correção final, mas sim a frustração e o tempo gasto.

Uma preocupação relacionada é que nunca pedimos aos alunos que escrevam explicitamente o parâmetro funcional. Embora isso não afete nossas questões de pesquisa, impacta a capacidade dos alunos de transitar de uma compreensão comportamental para o uso efetivo dessas funções de ordem superior. Isso pode ser especialmente relevante em situações como a de dobra, onde, ao final do semestre, muitos alunos podem "saber" que todas essas são instâncias de dobra, mas alguns deles podem não ser capazes de usar a dobra para produzir o comportamento descrito.

8 DISCUSSÃO

Questões de pesquisa. Nossas questões de pesquisa buscavam entender se os alunos estavam familiarizados com o espaço de características dos HAFs (Helping Frameworks) e se conseguiam conectar exemplos de entrada/saída a HAFs específicos. Encontramos alguns erros básicos: os alunos não confundem ``map`` e ``filter``, que diferem em múltiplas características, incluindo a relação entre a lista de entrada e a lista de saída. Eles são mais propensos a confundir ``filter`` e ``take-while``, que diferem apenas em quais elementos são retidos na lista de saída. Vale ressaltar que nem todos os desenvolvedores conseguem fazer essa distinção: como apenas um exemplo, um usuário ativo do StackOverflow...

O usuário [22], que se autodeclara "Desenvolvedor Java" com alta pontuação de reputação (em 14/03/2021), percebe [23] a semelhança no tipo e, em um exemplo específico, no comportamento entre essas duas funções e pergunta: "Qual era a necessidade desta nova função, então?" É possível encontrar diversas perguntas semelhantes na Web [27]. (Para ser justo, esses desenvolvedores podem estar acostumados a pensar na simulação imperativa de take-while usando filter.)

De modo geral, nossos dados sugerem que uma grande parcela dos alunos participantes consegue se sair razoavelmente bem nos testes de raciocínio espacial, inicialmente com pouco treinamento e avaliação, seguidos por algumas iterações de nossos instrumentos. Em particular, embora possam ter se beneficiado da familiaridade por meio da repetição, é fundamental observar que eles não receberam nenhuma avaliação individual que orientasse suas correções (exceto na Etapa 5, quando já teriam recebido feedback no contexto do curso). Presume-se que qualquer correção tenha sido feita a partir de sua própria observação.

Ainda observamos vários alunos cometendo erros de rotulagem que não são consistentes com os tipos, além de problemas com alunos que desconhecem a aplicabilidade do operador ``fold``. Uma questão fundamental é até que ponto esses problemas causam dificuldades durante a programação em si.

Erros de tipo seriam sinalizados em tempo de compilação ou de execução. A falha em identificar um possível uso de ``fold`` pode não levar a um código incorreto, apenas a um código mais longo. Nosso projeto geral ainda precisa abordar como os problemas que detectamos por meio de nossos instrumentos de estudo se manifestam no comportamento, na produtividade ou na experiência de programação. Isso seria melhor feito com um componente separado que observasse o que os alunos fazem enquanto programam.

As descrições de clusters feitas pelos alunos em formato livre nas duas primeiras etapas mostram que muitos alunos têm dificuldade em fornecer descrições concisas do comportamento de hofs. Essa não é uma observação nova: alunos, assim como programadores, podem ter dificuldade em escrever especificações comportamentais precisas [2, 17]. Se o ensino direto sobre como descrever hofs por meio de características ajudaria os alunos a desenvolver essa habilidade é uma questão em aberto.

Reflexão sobre o desenho do nosso estudo. Este é o primeiro estudo que vimos sobre o desenvolvimento da compreensão dos sistemas de habitat. Como tal, as lições relativas à nossa metodologia constituem, por si só, uma contribuição deste artigo. Sabendo o que sabemos agora, que mudanças faríamos nos nossos instrumentos e métodos?

- Começaríamos com a tarefa de classificação em vez da tarefa de agrupamento. A tarefa de classificação concentra-se em quais características dos objetos de uso da terra os alunos reconhecem. A tarefa de agrupamento, em contraste, exige que os alunos percebam as características e decidam como agrupá-las. Essa fusão torna mais difícil para nós distinguir quais concepções levaram às decisões de agrupamento. (Além disso, Barsalou [3] aponta considerações adicionais para o projeto de tarefas de categorização.) • Solicitaríamos

justificativas para as classificações em alguns exemplos. Nosso projeto atual, que apenas pede aos alunos que rotulem os exemplos, não nos informa quais características os alunos observaram em um exemplo (especialmente em um que múltiplas características surgem). Inferimos as características a partir dos hofs selecionados por um aluno, mas os alunos podem não estar pensando ativamente sobre as características ao fazer essas seleções. • Incluiríamos uma atividade na qual os alunos escrevessem exemplos ilustrativos dos hofs por si mesmos. Reconhecer características dentro de um determinado exemplo de entrada/saída é uma atividade diferente.

do que ser capaz de gerar exemplos de entrada/saída que reflitam as características definidoras que distinguem o HAF. Estamos curiosos para saber quais das características do HAF os alunos...

Abordar o assunto caso seja solicitado que eles gerem os exemplos por conta própria.

Estamos também repensando o uso dos diagramas da Etapa 3, visto que eles não parecem ter melhorado muito o desempenho dos alunos. Os diagramas resumem explicitamente os tipos e as características de extensão dos hofs que incluímos neste estudo. A possibilidade de transformação dos elementos está implícita nos tipos, mas os alunos podem não reconhecer essa característica sem indicadores adicionais, como formas, nos diagramas. Do ponto de vista metodológico, seria útil saber quais características os alunos percebem nos diagramas para que pudéssemos escolher um papel apropriado para esses diagramas tanto nos instrumentos de estudo quanto na pedagogia relacionada aos hofs.

Tipos Estáticos. Antes dos Estágios 1 a 4, os alunos foram expostos a funções de ordem superior (HOFs) por meio da versão dinamicamente tipada do Racket. Após a disciplina Como Projetar Programas, os alunos visualizaram as especificações de tipo como comentários, mas elas não foram verificadas. Portanto, os alunos não tiveram o benefício dos tipos estáticos. Durante o semestre, eles utilizaram o Pyret, uma linguagem com um verificador de tipo estático opcional, cujo uso foi, pelo menos brevemente, obrigatório. Não está claro como nossos resultados mudariam se os alunos tivessem trabalhado em uma linguagem tipada desde o início, especialmente considerando as relações matematicamente interessantes entre os tipos e o conjunto de comportamentos permitidos do programa [28].

Observamos relativamente pouca confusão quando os tipos de funções são suficientemente distintos: por exemplo, os alunos raramente confundem `map` com `filter` ou `take-while` (mas confundem rotineiramente as duas últimas). No entanto, esse fenômeno pode não ser generalizável: em nosso estudo, os alunos trabalharam com um número muito pequeno de funções de ordem superior (hofs), então poderiam facilmente ter simplesmente enumerado todas as que conheciam. Em linguas reais, pode haver dezenas dessas funções, várias com tipos sobrepostos e, portanto, funcionalidades semelhantes, mas distintas. Nosso estudo sugere que, pelo menos para iniciantes, é necessário um esforço muito maior para distinguir as funções que são semelhantes em tipo.

Da categorização às concepções errôneas. Os processos de categorização que utilizamos nos dizem apenas até certo ponto. Eles podem apontar tendências gerais e, em particular, identificar fragilidades comuns. Mas, sem mais relatos dos alunos, eles não podem nos dizer por que fizeram essas escolhas.

Dado que este foi um estudo pioneiro nesta área, não tínhamos certeza do esforço que exigiria dos alunos e, devido à COVID-19, queríamos manter as etapas com uma carga de trabalho especialmente baixa. Um estudo futuro, no entanto, deveria aprofundar a investigação sobre o raciocínio por trás das escolhas dos alunos. Esperamos que isso revele uma rica fonte de informação, mesmo nos casos em que escolheram a resposta "certa" (mas por razões "erradas"). Acreditamos que, como comunidade de pesquisa, ainda não compreendemos muito bem as concepções e concepções errôneas dos alunos sobre fluxos de informação. Seria também interessante relacionar as descobertas neste nível com aquelas em uma escala maior, como a análise de fluxos Java de Khatchadourian et al. [19].

Implicações para o Planejamento. Este artigo foi parcialmente motivado pelo valor do planejamento em estilos contemporâneos de programação, especialmente código centrado em dados. Alguns trabalhos anteriores de planejamento [9, 10] mostram sucesso quando os alunos estão familiarizados com hofs, que por sua vez Wickham mostra [29] se conectam bem à manipulação de dados padrão.

No entanto, nenhum desses trabalhos se concentrou no que é necessário para que os alunos se sintam à vontade com essas operações e desenvolvam uma boa "sensibilidade" para elas. Nosso artigo sugere técnicas para ajudá-los a compreender esses comportamentos.

Além de corrigir as falhas descritas acima, ainda faltam (pelo menos) duas partes neste trabalho para tornar os alunos melhores no planejamento:

- Por definição, as funções de ordem superior (hofs) são especificações comportamentais incompletas: muito depende do parâmetro da função que lhes é passado. Nosso trabalho ignorou esse parâmetro, pedindo aos alunos que imaginassem quais parâmetros poderiam ser passados. Focar no parâmetro da função — por exemplo, pedindo aos alunos que descrevam a função que mapearia uma determinada entrada para a saída desejada — deve gerar novas perspectivas sobre a compreensão das hofs pelos alunos. Conjecturamos que alguns alunos podem identificar corretamente qual hof realiza uma transformação, mas podem ter dificuldade em descrever o parâmetro funcional; será que os alunos também teriam dificuldade no sentido inverso?
- Este artigo focou-se especificamente em uma função de cada vez, ignorando a composição de funções. Quando começamos a considerar composições, o espaço de respostas possíveis para nossas tarefas de agrupamento e classificação torna-se muito mais rico (e, portanto, talvez muito mais difícil de gerenciar). No entanto, a composição de funções é fundamental para atingir o objetivo do planejamento. Assim, precisamos de métodos que possam lidar com essa complexidade e nos permitam ajudar os alunos a desenvolver concepções comportamentais das composições de funções (de ordem superior ou não).

AGRADECIMENTOS

Este trabalho foi parcialmente financiado pela Fundação Nacional de Ciência dos EUA. Somos especialmente gratos aos alunos de CSCI 0190, que se empenharam com entusiasmo nestes exercícios. Agradecemos a Rob Goldstone pelas conversas instrutivas. Elijah Rivera e nossos revisores forneceram comentários perspicazes que nos ajudaram a esclarecer nossa apresentação.

REFERÊNCIAS

- [1] Harold Abelson e Gerald Jay Sussman. 1985. Estrutura e interpretação de programas de computador. MIT Press, Cambridge, MA.
- [2] VL Almqvist. 1996. Investigando dificuldades dos alunos com lógica matemática. Em Ensino e Aprendizagem de Métodos Formais. Academic Press, 131–160.
- [3] Lawrence W. Barsalou. 1994. Derivando categorias para atingir objetivos. Psicologia da aprendizagem e motivação 27 (1994).
- [4] Michael J. Clancy e Marcia C. Linn. 1999. Padrões e Pedagogia. Nos Anais do Trigesimo Simpósio Técnico SIGCSE sobre Educação em Ciência da Computação (Nova Orleans, Louisiana, EUA) (SIGCSE '99). Association for Computing Machinery, Nova York, NY, EUA, 37–42. <https://doi.org/10.1145/299649.299673> [5] Jeffrey Dean e Sanjay Ghemawat. 2004. MapReduce: Processamento de Dados Simplificado em Grandes Clusters. Nos Anais da 6ª Conferência sobre Projeto e Implementação de Sistemas Operacionais - Volume 6 (São Francisco, CA) (OSDI'04). Associação USENIX, EUA, 10.
- [6] Ekwa Duala-Ekoko e Martin P. Robillard. 2012. Perguntando e respondendo perguntas sobre APIs desconhecidas: um estudo exploratório. Em Anais da Conferência Internacional de Engenharia de Software. 266–276.
- [7] K. Duncker. 1945. Sobre a resolução de problemas. Monografias Psicológicas 58, 5 (1945).
- [8] Matthias Felleisen, Robert Bruce Findler, Matthew Flatt e Shriram Krishnamurthi. 2001. Como projetar programas. MIT Press. <http://www.htdp.org/> [9] Kathi Fisler. 2014. O problema da chuva recorrente. Em Anais do ICER.
- [10] Kathi Fisler, Shriram Krishnamurthi e Janet Siegmund. 2016. Modernizando os estudos de composição de planos. Em Simpósio Técnico da ACM sobre Educação em Ciência da Computação.

- [11] Matthew Flatt e PLT. 2010. Referência: Raquete. Relatório Técnico PLT-TR2010-1. PLT Inc. <http://racket-lang.org/tr1/>.
- [12] Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. 1995. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, Reading, MA.
- [13] D. Gentner e K. Kurtz. 2005. Categorias relacionais. Em Categorização dentro e fora do laboratório. APA, Washington, DC, 151–175.
- [14] Eleanor J. Gibson. 1969. Princípios de aprendizagem e desenvolvimento perceptual. Appleton-Century-Crofts.
- [15] James J. Gibson e Eleanor J. Gibson. 1955. Aprendizagem perceptual: diferenciação ou enriquecimento? *Psychological Review* 62, 1 (1955), 32–41.
- [16] Andy Gordon e Simon Peyton Jones. 2021. LAMBDA: A função definitiva da planilha do Excel. <https://www.microsoft.com/en-us/research/blog/lambda-the-ultimate-excel-worksheet-function/>.
- [17] Geoffrey L. Herman, Michael C. Loui, Lisa Kaczmarczyk e Craig Zilles. 2012. Descrevendo o que e por que os alunos encontram dificuldades em lógica booleana. *ACM Transactions on Computing Education* 12, 1, Artigo 3 (março de 2012), 28 páginas. <https://doi.org/10.1145/2133797.2133800>
- [18] Graham Hutton. 1999. Um tutorial sobre a universalidade e expressividade da dobra. *Revista de Programação Funcional* 9, 4 (julho de 1999), 355–372.
- [19] Raffi Khatchadourian, Yiming Tang, Mehdi Bagherzadeh e Baishakhi Ray. 2020. Um estudo empírico sobre o uso e o mau uso de Streams no Java 8. Em *Abordagens Fundamentais para Engenharia de Software*.
- [20] KJ Kurtz e G. Honke. 2020. Resolvendo o problema do conhecimento inerte: construção de categorias para promover a transferência espontânea. *Journal of Experimental Psicologia: Aprendizagem, Memória e Cognição* 46, 5 (2020), 803–821.
- [21] Peter L. Pirolli. 1985. Resolução de problemas por analogia e aquisição de habilidades no domínio da programação. Dissertação de doutorado. Universidade Carnegie Mellon, Departamento de Psicologia Cognitiva.
- [22] Postagem do StackOverflow. [s.d.]. <https://stackoverflow.com/users/706317/zhekakozlov>.
- [23] Postagem no StackOverflow. [s.d.]. <https://stackoverflow.com/questions/46850689/how-is-takewhile-different-from-filter>.
- [24] Elliot Soloway. 1986. Aprender a programar = Aprender a construir mecanismos e explicações. *Commun. ACM* 29, 9 (set. 1986), 850–858.
- [25] Kyle Thayer, Sarah E. Chasins e Amy J. Ko. 2021. Uma teoria do conhecimento robusto de API. *ACM Transactions on Computing Education* 12, 1 (janeiro de 2021).
- [26] Simon Thompson. 1999. Haskell: A Arte da Programação Funcional (2ª ed.). Addison-Wesley.
- [27] Tópico do fórum MSDN. [nd]. <https://social.msdn.microsoft.com/Forums/en-US/393da5e4-f33b-4f0d-bfc1-ae73d2cd77df/what-is-the-difference-between-takewhile-and-where-in-linq>.
- [28] Philip Wadler. 1989. Teoremas de graça!. In *Anais da Quarta Conferência Internacional sobre Linguagens de Programação Funcional e Arquitetura de Computadores* (Imperial College, Londres, Reino Unido) (FPCA '89). Association for Computing Machinery, Nova York, NY, EUA, 347–359. <https://doi.org/10.1145/99370.99404>
- [29] Hadley Wickham. 2014. *Avançado R*. Chapman e Hall/CRC.
- [30] Hadley Wickham. 2019. A alegria da programação funcional (para ciência de dados). <https://www.youtube.com/watch?v=bzUmK0Y07ck>.